Posibles soluciones a los ejercicios del parcial práctico del 5-10-26

Importante: las soluciones que se muestran a continuación no son las únicas que se pueden considerar correctas para los ejercicios planteados.

Tema 1

1. Un sistema de seguridad debe analizar un conjunto de 10000 paquetes de red capturados, disponibles en una estructura de datos compartida. Para ello, dispone de W workers, los cuales trabajan colaborativamente analizando de a 1 paquete por vez cada uno. Cada análisis puede tomar un tiempo diferente, y para realizarlo los workers disponen de la función Clasificar(p), la cual retorna el protocolo al que pertenece el paquete (un número entero entre 0 y 15). Al finalizar el procesamiento, uno de los worker debe informar la cantidad de paquetes procesados por cada protocolo y qué worker procesó más paquetes. Implemente una solución usando **SEMÁFOROS**. **Nota**: maximizar la concurrencia.

```
sem sem_cola = 1;
sem sem_cont[16] = [16]{1};
sem sem_seq = 1;
sem s_bar = 1;
sem s_dormir = 0;

Queue paquetes; // ya cargada con 10000 paquetes
int contprot[16] = [16]{0};
int contworker[W] = [W]{0};
int cant = 0;

Process Worker [i=1..W] {
      int maxW, maxContW = 0;
      P(sem_cola);
      while (not empty(paquetes)) do
            p = pop(paquetes);
            V(sem_cola);

            protocolo = Clasificar(p);

            // sección crítica por posición ---
            P(sem_cont[protocolo]);
            contprot[protocolo]++;
            V(sem_cont[protocolo]);

            // no requiere protección
            contworker[i]++;

            P(sem_cola);
       end;
       V(sem_cola);

       // barrera final
       P(s_bar);
       cant = cant + 1;
       if (cant < W) then
            V(s_bar);
            P(s_dormir);
       else
             // informar cantidad de paquetes por protocolo
            for i := 0 to 9 do
                  print(i, contprot[i]);
            // informar worker que más paquetes procesó
```

```
            for i := 0 to W do
                  if (contworker[i] > maxContW ? contworker[i]) then
                        maxContW := contworker[i];
                        maxW = i;
                  end
            end
            print(maxW);
            for i := 1 to W-1 do
                  V(s_dormir);
            V(s_bar);
      end;
}
```

Comentarios:

- Algunos pueden haber llevado la cuenta de forma local y recién actualizar los contadores de cada contprot al salir del while.
- Algunos pueden haber usado una función para calcular e imprimir (sobre todo el último print) pasando como parámetro contworker
- En la barrera final, no es necesario que se duerman los worker hasta que se haya impreso todo

2. Existen N bandas que deben usar la sala de ensayo de una escuela de música. La sala sólo puede ser usada por una banda a la vez, dando prioridad de acuerdo con la antigüedad de la banda (cuando la sala está libre la debe usar la banda con más años de trayectoria entre las que están esperando). Implemente una solución con **MONITORES** que considere únicamente procesos Banda. **Nota**: la función obtenerAntiguedad(i) retorna la antigüedad para la banda i.

```
Monitor Sala {
      int usando = 0, esperando = 0;
      cond colas[N];
      Queue(int,int) espera; // (antigüedad, id), se inserta ordenado por
antigüedad desc
      procedure pedir (int id, int antiguedad) {
            if (usando > 0) then
                  esperando++;
                  pushOrdenado(espera, (antiguedad, id));
                  wait(colas[id]);
            else
                  usando++;
      }

      procedure liberar () {
            if (esperando > 0) then
                  int id = popMax(espera);
                  signal(colas[id]);
                  esperando--;
            else
                  usando--;
      }
}

Process Banda [i:1..N] {
      int antiguedad = obtenerAntiguedad();
      Sala.pedir(i, antiguedad);
      Ensayar();
      Sala.liberar();
}
```

Tema 2

1) Existen N bandas que deben usar la sala de ensayo de una escuela de música. La sala sólo puede ser usada por una banda a la vez, dando prioridad de acuerdo con la antigüedad de la banda (cuando la sala está libre la debe usar la banda con más años de trayectoria entre las que están esperando). Implemente una solución con **SEMÁFOROS** que considere únicamente procesos Banda. **Nota**: la función *obtenerAntiguedad(i)* retorna la antigüedad para la banda *i*.

```
Alternativa 1:

Queue cola;
sem mutex = 1,
espera[N] = ([N] 0);
boolean libre = true;

Process Banda[id: 0..N-1]{
      int antiguedad = obtenerAntiguedad();
      int aux;
      // llega a la sala
      P (mutex);
      if (libre) then
            libre = false;
            V (mutex);
      else
            // encola id ordenado por antiguedad
            pushOrdenado (cola, (id,antiguedad));
            V (mutex);
            P (espera[id]);
      end
      // usa la sala
      Ensayar();
      // sale de la sala
      P (mutex);
      if (empty (cola)) then
            libre = true;
      else
            pop (cola, aux); // desencola id
            V (espera[aux]);
      end
      V(mutex);
}
```

Alternativa 2:

```
Queue cola;
sem mutex = 1,
espera[N] = ([N] 0);
boolean libre = true;

Process Banda[id: 0..N-1]{
      int antiguedad = obtenerAntiguedad();
      int aux;
      // llega a la sala
      P (mutex);
      if (not libre) then
            // encola id ordenado por antiguedad
            pushOrdenado (cola, (id,antiguedad));
            V (mutex);
            P (espera[id]);
      end
    libre = false;
      V (mutex);
      // usa la sala
      Ensayar();
      // sale de la sala
      P (mutex);
      libre = true;
      if (empty (cola)) then
            V(mutex);
      else
            pop (cola, aux); // desencola id
            V (espera[aux]);
      end
}
```

2. Una aplicación de viajes compartidos junta a pasajeros de a 4 antes de salir. Existen P pasajeros (P múltiplo de 4). Cada pasajero se anota en la aplicación, y en cuanto se completan 4 anotados, entonces forman un grupo y suben al auto para salir de viaje (el viaje dura un tiempo fijo de 30 min). Al terminar el viaje, los 4 pasajeros del grupo se bajan. Implemente una solución usando **MONITORES**. **Nota**: maximizar la concurrencia.

```
Process Pasajero [i:1..P] {
      int miGrupo;
      ArmadoDeGrupo.anotarse(miGrupo);
      "Subir al auto";
      Viaje[miGrupo].viajar();
      "Bajar del auto";
}

Monitor ArmadoDeGrupo {
      int cant = 0;
      int grupoActual = 0;
      cond pasajeros[P/4];

      procedure anotarse (var int miGrupo) {
            miGrupo = grupoActual;
            cant++;
            if (cant < 4) then
                  wait(pasajeros[miGrupo]);
            else
                  cant = 0;
                  grupoActual++;
                  signal_all(pasajeros[miGrupo]);
            end
      }
}

Monitor Viaje [i=0..P/4-1] {
      cond pasajeros;
      int cant = 0;

      procedure viajar () {
            cant++;
            if (cant < 4) then
                  wait(pasajeros);
            else {
                  delay(30*60); "Viajar"
                  signal_all(pasajeros);
            }
      }
}
```